

Pipeline pratica: da YOLO al grafo

Scaletta operativa della prima versione del sistema

Obiettivo pratico della prima versione

- rilevare i componenti principali tramite YOLOv11 exp11b
- assegnare un'identità univoca a ogni componente nel singolo schema
- stimare i terminali dei componenti più semplici
- estrarre fili o segmenti di collegamento
- ricostruire le **net** principali
- associare i terminali alle net
- salvare il risultato in una struttura dati convertibile in grafo es json

Idea guida

- La prima versione deve essere semplice ma utile.
- Non serve riconoscere subito tutti i componenti.
- Non serve leggere subito tutto il testo.
- Questa prima versione è un **MVP**: una pipeline minima ma già significativa.

Scelta iniziale

- partire da diagrammi relativamente semplici
- Inizialmente considerare solo **componenti a 2 terminali**
- escludere temporaneamente componenti più complessi
- non usare ancora OCR: il testo viene trattato solo come possibile disturbo visivo

1. Rilevazione dei componenti con YOLO

Input e modello

- Input: una singola immagine del diagramma.
- Modello usato: YOLOv11 exp11b, già individuato come il migliore tra quelli addestrati.
- La detection dei componenti viene considerata il punto di partenza.

Output minimo per ogni detection

- class_id
- confidence
- bounding box
- image_id

Significato

- Il primo output concreto è una lista strutturata di componenti rilevati, con posizione e classe di appartenenza.
- Da questo momento in poi YOLO non è più il centro del lavoro: diventa il primo blocco della pipeline.

Esempio di output iniziale

```
{
  "image_id": "schema_001",
  "components": [
    {"class_id": 22, "conf": 0.94,
     "bbox": [100, 200, 180, 240]},
    {"class_id": 22, "conf": 0.91,
     "bbox": [240, 200, 320, 240]},
    {"class_id": 4, "conf": 0.96,
     "bbox": [410, 250, 470, 300]}
  ]
}
```

2. Dalle classi alle istanze univoche

Problema

- Se YOLO trova più oggetti della stessa classe, questi non possono rimanere con lo stesso identificatore.
- Nel grafo finale ogni componente deve essere distinguibile dagli altri.

Regola di rinomina

- il primo componente di classe 22 diventa 22.1
- il secondo componente di classe 22 diventa 22.2
- il terzo componente di classe 22 diventa 22.3
- il primo componente di classe 4 diventa 4.1, e così via

Esempio di componenti dopo la rinomina

```
{
  "image_id": "schema_001",
  "components": [
    {"instance_id": "22.1",
     "class_id": 22,
     "bbox": [100, 200, 180, 240],
     "conf": 0.94},
    {"instance_id": "22.2",
     "class_id": 22,
     "bbox": [240, 200, 320, 240],
     "conf": 0.91},
    {"instance_id": "4.1",
     "class_id": 4,
     "bbox": [410, 250, 470, 300],
     "conf": 0.96}
  ]
}
```

Perché è importante

- Si passa dal semplice riconoscimento di “tipi di oggetti” alla definizione di componenti realmente distinguibili nel singolo circuito.
- Questa tabella è la base di tutta la pipeline successiva.

Tabella strutturata per ogni oggetto

- ID istanza
- classe YOLO
- bounding box
- score di confidenza
- immagine di appartenenza

3. Definizione dei terminali per classe

Perché servono i terminali

- Per ricostruire un circuito non basta sapere dov'è il simbolo nel diagramma.
- Bisogna sapere in quali punti il simbolo entra in contatto con i fili.

Libreria di metadati delle classi

- nome del componente
- numero di terminali
- posizione attesa dei terminali rispetto al bounding box del simbolo

Scelta per la prima fase

- Partire da componenti semplici a 2 terminali.
- Posizione dei terminali definita in modo semplice e approssimato, usando punti relativi al bounding box

Esempio: class_terminals.yaml

```
22:
  name: resistor
  terminals:
    - name: t1
      relative_position: left
    - name: t2
      relative_position: right
4:
  name: capacitor
  terminals:
    - name: t1
      relative_position: left
    - name: t2
      relative_position: right
```

Output di esempio

```
{
  "instance_id": "22.1",
  "class_id": 22,
  "bbox": [100, 200, 180, 240],
  "terminals": [
    {"terminal_id": "22.1:t1", "x":
100, "y": 220},
    {"terminal_id": "22.1:t2", "x":
180, "y": 220}
  ]
}
```

4. Estrazione dei fili dal diagramma

Obiettivo

- Ottenere una rappresentazione dell'immagine in cui rimangano soprattutto i fili.
- Ridurre il più possibile le interferenze dovute ai simboli dei componenti.

Procedura operativa

1. prendere l'immagine originale del diagramma
2. mascherare temporaneamente le regioni occupate dai bounding box dei componenti rilevati
3. convertire l'immagine in scala di grigi o in formato binario
4. applicare operazioni di pulizia del rumore
5. isolare le linee del circuito
6. applicare **closing** e **thinning/skeletonization** per migliorare la continuità e ridurre i fili a una traccia sottile

Cosa si ottiene

- immagine “ripulita” dei fili
- immagine skeleton dei collegamenti
- base per la ricostruzione della topologia

Schema del flusso

```
immagine originale  
  -> maschera  
componenti  
  -> binarizzazione  
  -> pulizia rumore  
  -> linee isolate  
  -> closing  
  -> thinning /  
skeletonization
```

Nota metodologica

- Questa parte segue la logica del **connection layer del paper**: *connected component analysis, closing e thinning per ricostruire i fili e migliorarne la continuità anche in presenza di interruzioni o occlusioni.*

Perché è un passaggio delicato

- Da qui in poi il problema non è più “trovare i simboli”, ma capire dove passano davvero i collegamenti del circuito.
- **Closing**: chiudere piccoli buchi o piccole interruzioni nelle linee
 - Unisce segmenti vicini
 - Riempie piccoli spazi
 - Rende fili continui
- **Thinning**: assottigliare le linee e portarle a uno spessore minimo
- **Skeletonization**: ridurre il filo al suo scheletro centrale. Utile per trovare:
 - Endpoint, giunzioni, intersezioni e componenti connessi

5. Ricostruzione delle net

Obiettivo

- Capire quali segmenti appartengono alla stessa rete elettrica e quali invece sono separati.

Elementi da identificare

- **connected components** delle linee
- **endpoint** dei segmenti
- **punti di giunzione**
- **intersezioni**
- eventuali **nodi comuni** del circuito

Risultato atteso

- Una collezione di **net**, identificate ad esempio come N1, N2, N3, N4.
- Ogni net rappresenta un insieme continuo di linee collegate tra loro.

Output possibile

```
{
  "nets": [
    {"net_id": "N1", "pixels":
[...]}},
    {"net_id": "N2", "pixels":
[...]}},
    {"net_id": "N3", "pixels":
[...]}
  ]
}
```

Interpretazione

- Non si tratta più solo di “vedere i fili”.
- Bisogna capire **quali pixel** e **quali segmenti** fanno parte della stessa struttura elettrica.

Aspetto importante

- La ricostruzione corretta delle net dipende anche dal riconoscimento delle giunzioni complesse, come **T-junction** e **X-junction**.

6. Associazione terminale-net

Input necessari

- componenti con i rispettivi terminali
- net ricostruite dai fili

Problema centrale

- Determinare quale terminale è collegato a quale net.
- Per ogni terminale stimato si verifica quale net tocca o passa in prossimità di quel punto.

Tabella di associazione attesa

- 22.1:t1 -> N1
- 22.1:t2 -> N2
- 4.1:t1 -> N2
- 4.1:t2 -> N3

Output possibile

```
{
  "connections": [
    { "terminal_id": "22.1:t1",
      "net_id": "N1" },
    { "terminal_id": "22.1:t2",
      "net_id": "N2" },
    { "terminal_id": "4.1:t1",
      "net_id": "N2" },
    { "terminal_id": "4.1:t2",
      "net_id": "N3" }
  ]
}
```

Perché è il cuore della pipeline

- Qui avviene il passaggio da immagine a struttura circuitale.
- Una volta nota l'associazione terminale-net, si può dedurre che un componente è connesso tra due o più net del circuito.

Esempio di deduzione

- Se 22.1:t1 è su N1 e 22.1:t2 è su N2, allora 22.1 è un componente connesso tra N1 e N2.

7. Costruzione del primo grafo

Quando è possibile costruirlo

- Quando sono disponibili componenti, terminali, net e associazioni terminale-net.

Struttura del grafo

- un nodo Diagram
- nodi Component
- nodi Terminal
- nodi Net

Relazioni principali

- Diagram -> HAS_COMPONENT -> Component
- Component -> HAS_TERMINAL -> Terminal
- Terminal -> CONNECTED_TO -> Net

Output atteso

```
{
  "diagram_id": "schema_001",
  "components": [...],
  "terminals": [...],
  "nets": [...],
  "edges": [
    {"from": "22.1", "to":
"22.1:t1", "type":
"HAS_TERMINAL"},
    {"from": "22.1:t1", "to":
"N1", "type": "CONNECTED_TO"}
  ]
}
```

Perché il grafo passa attraverso le net

- È preferibile a un collegamento diretto componente-componente, perché dal punto di vista elettrico è più corretto far passare la connessione attraverso la net.

Esempio

- Se 22.1:t2 e 22.2:t1 sono entrambi collegati alla net N2, allora i due componenti condividono quella stessa rete elettrica.

8. Strategia consigliata di sviluppo

Fase A: MVP iniziale

- utilizza YOLO per rilevare i componenti
- assegna gli ID istanza
- lavora solo su classi semplici a 2 terminali
- estrae i fili principali
- costruisce le net
- collega terminali e net
- salva una prima versione del grafo

Fase B: aumento di complessità

- **componenti con più di 2 terminali**
- mascheramento più accurato del testo
- OCR (forse)
- associazione tra testo e componente
- controlli topologici più raffinati

Fase C: versione completa

- grafo completo del diagramma
- query sul circuito
- eventuale esportazione verso **Neo4j** o altri graph database
- arricchimento semantico con refdes e valori letti tramite OCR

Primo obiettivo concreto di implementazione

- risolvere con successo un caso semplice su una singola immagine
- ottenere componenti, istanze, terminali, fili principali, alcune net e alcune associazioni corrette
- salvare il risultato in una struttura dati assimilabile a un grafo

9. Sequenza pratica di lavoro

Scaletta operativa

- 1. selezionare una **singola immagine semplice**
- 2. eseguire **YOLOv11 exp11b** sull'immagine
- 3. trasformare le classi rilevate in **istanze univoche**
- 4. definire i terminali per classi semplici
- 5. mascherare i componenti e isolare i fili
- 6. applicare **binarizzazione, pulizia e skeletonization**
- 7. individuare le **net principali**
- 8. associare i terminali alle net
- 9. salvare il risultato in formato JSON o altra struttura intermedia
- 10. convertire successivamente tale struttura in un primo grafo

Sintesi finale

- Il lavoro parte da un caso pilota molto semplice.
- L'obiettivo non è ricostruire subito l'intero circuito, ma verificare che ogni blocco della pipeline funzioni correttamente.
- Una volta validata questa prima catena, sarà possibile estendere il metodo a più immagini e a componenti più complessi.

Flusso generale

- Immagine -> YOLO -> componenti -> istanze
- -> terminali -> fili -> net -> collegamenti -> grafo

Risultato atteso della prima implementazione

- Trasformare il diagramma da semplice immagine annotata a struttura logica interrogabile.